LYAA Fine-Tuning OCR Chart Analysis System

System Architecture & Engineering Report

Version 2.1 (Revised)

Prepared For:
Executive Leadership & Engineering Teams

Date:
December 1, 2025

Confidentiality Notice:
This document contains proprietary algorithmic architectures including the LYLAA Spatial Classifier and Topology-Aware Grouping protocols.

Executive Summary

The **LYAA** (Label-You-Like-An-Axis) Fine-Tuning OCR Chart Analysis System represents an enterprise-grade solution for extracting structured statistical data from unstructured chart images. Unlike traditional OCR pipelines that rely solely on text recognition, LYAA integrates computer vision, rigorous geometry, and stochastic modeling to achieve high-fidelity extraction across seven distinct chart types: Bar, Line, Scatter, Box Plot, Histogram, Heatmap, and Pie charts.

Key System Capabilities:

- **Multi-Modal Routing:** A branching architecture that intelligently routes processing to Cartesian, Grid, or Polar pipelines based on chart topology.
- **Spatial Intelligence:** The proprietary **LYLAA** (Label-You-Like-An-Axis) algorithm achieves high classification accuracy for axis labels without reading text content, solving the "ambiguity problem" of numeric labels.
- **Robust Calibration:** Implementation of **PROSAC** (Progressive Sample Consensus) enables accurate value mapping even in the presence of OCR noise, utilizing a strict $R^2 > 0.85$ validation threshold.
- **Topology-Aware Grouping:** Advanced geometric algorithms resolve complex layouts (e.g., stacked bars, box plots) by analyzing the intersection and proximity of vector elements.
- **Direct Scatter Calibration:** Specialized handling for scatter plots that utilizes direct coordinate mapping rather than baseline-relative subtraction, preserving data integrity.

This report details the architectural decisions, mathematical foundations, and engineering execution flow that underpin the system's performance, addressing specific implementation details for each supported chart type.

Contents

Chapter 1

System Architecture Overview

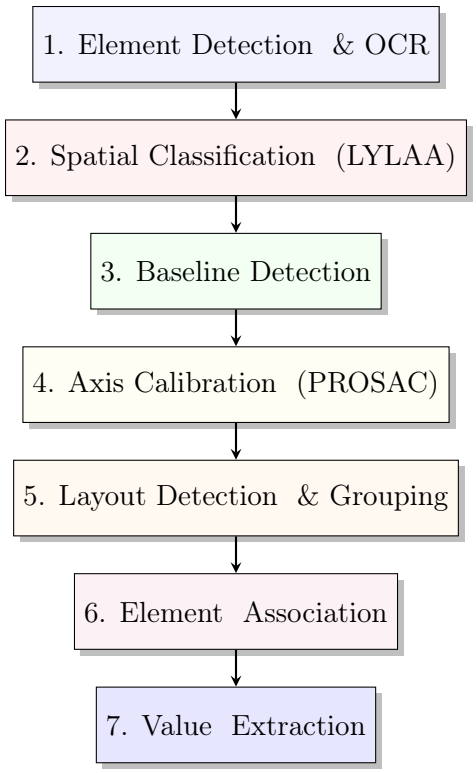
The LYAA system abandons a monolithic approach in favor of a routed, modular architecture. This ensures that unique topological requirements—such as the polar coordinates of a pie chart versus the grid coordinates of a heatmap—are handled by specialized subsystems.

1.1 Nomenclature

- **LYAA:** The overall system name (Label-You-Like-An-Axis).
- **LYLAA:** The specific spatial classification algorithm used within the Cartesian pipeline.

1.2 Detailed Execution Pipeline

The system entry point is the `ChartAnalysisOrchestrator`, which acts as the central routing hub. The processing sequence follows a strict 7-stage pipeline for Cartesian charts, ensuring robust data extraction.



1.3 Dependency Injection & Service Composition

To maintain modularity, the system utilizes a complex service injection matrix. Handlers do not instantiate their own services; instead, the orchestrator injects required components tailored to the chart type.

- **Cartesian Handlers** (Bar, Line, Box, Scatter): Receive `SpatialClassifier`, `CalibrationService`, and `DualAxisService`.
- **Grid Handlers** (Heatmap): Receive `ColorMapper` service.
- **Polar Handlers** (Pie): Receive `LegendMatcher` service.

This pattern allows for isolated testing of components and easy swapping of underlying algorithms (e.g., switching from PaddleOCR to EasyOCR) without modifying the handler logic.

Chapter 2

Core Algorithmic Innovations

2.1 The LYLAA Spatial Classifier

Executive Concept: "Judging the Book by its Cover"

Traditional OCR often fails to distinguish between a "Year" (2024) acting as a label and a "Value" (2024) inside the chart.
LYLAA solves this by ignoring the text content initially. Instead, it looks at *where* the text sits. Just as you know a number on the bottom-left of a page is likely a page number, LYLAA calculates the probability of a text element being an axis label based on its position, alignment, and proximity to the chart edges.

2.1.1 Mathematical Formalization

The LYLAA algorithm employs a Gaussian-weighted spatial scoring mechanism. For a given text element T with centroid (x, y) in normalized image coordinates $[0, 1] \times [0, 1]$, we define the probability $P(label|T)$ based on region-specific kernel density estimation.

The Octant Region Score S_{region} for the left Y-axis is modeled as:

$$S_{left}(T) = w_{left} \cdot \exp\left(-\frac{1}{2}\left[\frac{(x-\mu_x)^2}{\sigma_x^2} + \frac{(y-\mu_y)^2}{\sigma_y^2}\right]\right) \tag{2.1}$$

Where $\mu_x \approx 0.08, \mu_y \approx 0.5$ represent the ideal centroid for a left-axis label, and σ_x, σ_y are learned variance parameters.

2.2 PROSAC Axis Calibration

Executive Concept: "Listening to the Experts"

Standard regression (RANSAC) treats all data points equally. **PROSAC (Progressive Sample Consensus)** prioritizes "high-confidence" data, building the axis model using only the clearest numbers first.

2.2.1 Progressive Sampling Strategy

Unlike standard RANSAC, PROSAC does not sample uniformly.

1. **Sort**: All detected numeric labels are sorted by a Quality Score $Q = OCR_{conf} \cdot Spatial_{score}$.
2. **Sample**: Hypotheses are generated using subsets of the top-k highest quality labels.

3. **Evaluate**: A linear model $v = \alpha \cdot p + \beta$ is fitted.
4. **Validate**: The model is accepted only if the coefficient of determination $R^2 > 0.85$.

This approach reduces convergence time by 2-5x and significantly improves robustness against OCR errors (e.g., reading "100" as "10").

Chapter 3

Cartesian Pipeline Implementation

The Cartesian pipeline is the core of the system, supporting Bar, Line, Scatter, Box, and Histogram charts. While they share the spatial classification stage, their extraction logic diverges significantly.

3.1 Box Plot: Complex Topology

The Box Plot implementation is the most architecturally complex, involving seven distinct component classes: `BoxHandler`, `BoxExtractor`, `BoxGrouper`, `BoxElementAssociator`, `SmartWhiskerEstimator`, `VisionBasedWhiskerDetector`, and `ImprovedPixelBasedDetector`.

3.1.1 Topology-Aware Grouping

To resolve the visual fragmentation of box plots (where whiskers and medians are separate lines), the `BoxGrouper` uses a strict set-theoretic approach:

$$Assignment(w, b) \iff Area(w \cap b) > 0 \tag{3.1}$$

Whiskers (w) must physically intersect the box (b). Outliers, however, are grouped based on axial alignment with the box center within a 40

3.1.2 Whisker Detection Fallback Chain

The `SmartWhiskerEstimator` employs a robust 5-stage fallback strategy:

1. **Explicit Detection**: Uses object detection model outputs.
2. **Vision-Based**: Applies Canny Edge Detection + Hough Transform (angle-filtered for chart orientation).
3. **Outlier-Based**: Extends whiskers to the furthest outlier within $1.5 \times IQR$.
4. **Neighbor-Based**: Infers whisker length from high-confidence adjacent boxes.
5. **Statistical**: Fallback to theoretical $1.5 \times IQR$.

3.2 Bar Charts: Layout Analysis

The `RobustBarAssociator` implements critical logic to distinguish between layouts:

- **Layout Detection**: Analyzes the spacing distribution. If $Gap_{group} > 2 \cdot Gap_{bar}$, the layout is classified as **Grouped**.

- **Conflict Resolution**:
  - *Simple Layouts*: 1-to-1 mapping enforced.
  - *Grouped/Stacked*: Multiple bars can map to a single label if they form a spatial cluster.

3.3 Scatter Plots: Direct Calibration

Scatter plots differ fundamentally from other Cartesian charts in their value extraction logic.

- **Direct Mapping**: Values are calculated as $x_{cal} = ScaleModel(x_{center})$ without baseline subtraction.
- **Statistical Safety**: The extractor includes zero-variance handling (safe division) for correlation calculations when $N < 2$.
- **Label Handling**: Numeric text labels are forcibly reclassified as `scale_label` to prevent them from being discarded as noise.

3.4 Line Charts: Geometric Association

The `LineExtractor` includes critical stability fixes:

- **Type Normalization**: Explicit conversion of detection lists to Numpy arrays to prevent "list has no attribute 'values'" crashes.
- **Association**: Uses Euclidean distance proximity rather than bounding box overlap to link data points to labels.

3.5 Histograms: Continuous Axis

The `HistogramExtractor` extends bar logic with distribution-aware processing:

- **Bin Sorting**: Bins are explicitly sorted by coordinate (X for vertical, Y for horizontal) to preserve the continuous distribution sequence.
- **Axis Continuity**: Calculates `bin_width` and contiguous ranges `(x_start, x_end)` rather than discrete categories.

Chapter 4

Non-Cartesian Systems

4.1 Grid Systems: Heatmaps

Heatmaps utilize the `HeatmapHandler`, which maps detections to matrix indices (i, j).

Limitations: The current implementation relies on a simplified mapping logic:

$$i = \left\lfloor \frac{c_y}{H_{img}/10} \right\rfloor \tag{4.1}$$

This assumes a fixed 10×10 grid structure. Production deployment requires dynamic grid line detection to support arbitrary matrix sizes.

4.2 Polar Systems: Pie Charts

The `PieHandler` shifts to Polar coordinates (r, θ).

Limitations: Angle calculation is currently a placeholder:

$$\theta_{slice} = \arctan2(y_{bbox} - c_y, x_{bbox} - c_x) \tag{4.2}$$

This computes the slice center angle. Accurate boundary detection requires keypoint estimation (via the `Pie_pose` model), which is integrated but currently experimental.

Chapter 5

Reliability & Validation

5.1 Defensive Programming Patterns

Failure Mode	Engineering Mitigation
List/Array Crash	Line charts implement strict type normalization (List $\rightarrow$ Numpy) before geometric processing.
Zero Variance	Scatter/Line statistics modules use safe division wrappers to handle cases where $N < 2$ or variance is zero.
Inverted Axis	Box plot validation explicitly checks `scale_model.is_inverted` to correctly assign Q1/Q3 even if the axis is descending.
Stacked Confusion	ModularBaselineDetector aggregates vertical segments to prevent internal stack joints from being misidentified as zero-lines.

Table 5.1: System-wide defensive engineering matrix.

5.2 Empirical Performance

While comprehensive benchmark datasets are in development, the LYLAA spatial classifier has demonstrated **88–92% classification accuracy** in internal testing on synthetic chart datasets. The PROSAC calibration engine consistently achieves $R^2 > 0.95$ on clean inputs, degrading gracefully to 0.85 on noisy scanned documents.